

GitHub Actions

Complete Developer Interview & CI/CD Notes

.NET 8 • ASP.NET Core • SonarCloud • Code Coverage • Artifacts • Azure App Service • OIDC • NuGet • Reusable Workflows

A structured book-style consolidation of the GitHub Actions course covered in this conversation. It follows the same developer-focused approach as the Azure DevOps notes: concepts first, practical YAML second, enterprise architecture, troubleshooting concepts, and interview questions.

Sample: EmployeeManagementAPI (.NET 8)

Focus: practical .NET developer interview depth; unnecessary DevOps-admin detail is intentionally minimized.

Course Roadmap

Lesson	Topic	Core outcome
1	Fundamentals	Workflow, event, job, runner, step, action
2	First .NET CI	Build and test a .NET 8 application
3	Triggers	push, pull_request, manual, schedule, filters
4	Jobs, Runners & Steps	Execution model and dependencies
5	Conditions	if, status functions, needs
6	Variables & Contexts	env, vars, secrets, GitHub context
7	Secrets & Security	Secrets, environments, OIDC
8	Testing & Coverage	Test results and coverage
9	SonarCloud	.NET Begin/End analysis and Quality Gates
10	Artifacts	Build Once, Deploy Many
11	Enterprise CI	Source → validation → artifact
12	CD Fundamentals	Artifact promotion and environments
13	Azure App Service	OIDC and Azure deployment
14	Complete CD	Dev → QA → Production
15	Reusable Workflows	workflow_call and inputs
16	Matrix	Multiple configurations and parallelism
17	NuGet	Caching, Azure Artifacts, GitHub Packages

Lesson 1 — Fundamentals

What is GitHub Actions?

GitHub Actions is GitHub's automation platform for CI, CD, testing, security checks, scheduled jobs, package publishing, and repository automation.

Mental model

Workflow → Event → Job → Runner → Steps → Action / Command

Azure DevOps mapping

Azure DevOps	GitHub Actions
Pipeline	Workflow
Agent	Runner
Job	Job
Step	Step
Task	Action / run
Artifact	Artifact
Environment	Environment
condition	if
dependsOn	needs
trigger	on

Key idea

A workflow reacts to an event, creates jobs, and each job executes its steps on a runner.

Lesson 2 — First .NET CI

Goal

Validate EmployeeManagementAPI on a pull request by restoring, building, and testing the .NET 8 solution.

Initial CI YAML

```
name: Employee API - CI

on:
  pull_request:
    branches:
      - master

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout source code
        uses: actions/checkout@v4
      - name: Setup .NET
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: &#x27;8.0.x&#x27;
      - name: Restore NuGet packages
        run: dotnet restore
      - name: Build solution
        run: dotnet build --no-restore --configuration Release
      - name: Run unit tests
        run: dotnet test --no-build --configuration Release
```

Key syntax

uses invokes an action. **run** executes a shell command. **runs-on** selects the runner. Checkout makes source available.

Lesson 3 — Triggers

Main events

push — commits pushed to matching branches.

pull_request — PR validation.

workflow_dispatch — manual run.

schedule — cron execution.

workflow_run — react to another workflow completing.

Examples

```
on:
  push:
    branches:
      - master

on:
  pull_request:
    branches:
      - master

on:
  workflow_dispatch:
```

Filters

```
on:
  push:
    branches:
      - master
    paths:
      - &#x27;src/**&#x27;;
      - &#x27;tests/**&#x27;;
```

CI vs CD

PR validation commonly uses `pull_request`. Post-merge CI/CD can use `push` to master, or CD can be chained from successful CI with `workflow_run`.

Lesson 4 — Jobs, Runners & Steps

Execution model

```
Workflow
  ■■■ Job A → Runner A → steps
  ■■■ Job B → Runner B → steps
```

Steps in a job execute sequentially on the same runner. Independent jobs can run in parallel.

needs

```
jobs:
  test:
    needs: build
```

`needs` creates dependency/order. A dependent job normally waits for its upstream job.

Files between jobs

Runner workspaces are not automatically shared between jobs. Use artifacts or another explicit storage mechanism.

Lesson 5 — Conditions

Mental model

```
on:    When should the workflow start?
needs: When should the job run relative to another?
if:    Should this job/step run at all?
```

Examples

```
if: github.ref == &#x27;refs/heads/master&#x27;;
if: success()
if: failure()
```

```
if: always()
if: cancelled()
```

PR contexts

github.base_ref is the PR target branch; github.head_ref is the PR source branch.

Best practice

Prefer a clean dependency graph over complicated conditional logic.

Lesson 6 — Variables, Expressions & Contexts

Configuration mechanisms

env — environment variables. **vars** — non-sensitive configuration. **secrets** — sensitive values. Expressions use `${{ ... }}`.

Example

```
env:
  DOTNET_VERSION: &#x27;8.0.x&#x27;
  BUILD_CONFIGURATION: &#x27;Release&#x27;

steps:
- uses: actions/setup-dotnet@v4
  with:
    dotnet-version: ${{ env.DOTNET_VERSION }}
```

Contexts

github, env, vars, secrets, runner, job, steps, needs, strategy, matrix.

Useful values

github.ref, github.sha, github.actor, github.repository, github.event_name, github.base_ref, github.head_ref.

Azure mapping

```
$(Build.SourceBranch)    → ${{ github.ref }}
$(Build.SourceVersion)   → ${{ github.sha }}
$(Build.Repository.Name) → ${{ github.repository }}
$(Build.Reason)          → ${{ github.event_name }}
```

Lesson 7 — Secrets & Secure Configuration

Rules

Never hardcode credentials. Use repository, organization, or environment secrets according to scope. Do not print secrets to logs.

OIDC

OpenID Connect lets GitHub Actions authenticate to Azure using short-lived identity tokens instead of a long-lived Azure credential.

Flow

```
GitHub Actions
  ↓
OIDC identity token
  ↓
Microsoft Entra ID
  ↓
Azure RBAC
  ↓
Azure resource
```

Interview answer

OIDC reduces credential exposure because Azure trusts a configured GitHub identity and the workflow obtains a short-lived token.

Lesson 8 — Testing & Code Coverage

Testing vs coverage

Tests validate behavior. Coverage measures which code was executed. Coverage does not prove that tests are good or code is bug-free.

Generate

```
dotnet test --collect:"XPlat Code Coverage" --logger "trx";
```

Publish reporting

```
- name: Upload test results
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: test-results
    path: **/*.trx;

- name: Upload code coverage
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: code-coverage
    path: **/coverage.cobertura.xml;
```

Three concepts

Coverage generation, publishing, and enforcement are different. A Quality Gate or explicit check can enforce thresholds.

Practical approach

For legacy code, New Code Coverage is often more practical than immediately requiring high overall coverage.

Lesson 9 — SonarCloud Integration for .NET

Why SonarCloud?

It complements tests and coverage by identifying bugs, code smells, vulnerabilities, security hotspots, duplication, and maintainability issues.

Azure DevOps vs GitHub Actions

Azure DevOps exposes SonarCloudPrepare, SonarCloudAnalyze, and SonarCloudPublish tasks. For .NET in GitHub Actions, the common scanner lifecycle is Begin → Build/Test → End; SonarCloud then evaluates the resulting Quality Gate.

Flow

```
SonarScanner Begin
↓
Restore
↓
Build
↓
Test + Coverage
↓
SonarScanner End
↓
SonarCloud
↓
Quality Gate
```

Representative implementation

```
- name: Install SonarScanner
  run: dotnet tool install --global dotnet-sonarscanner
```

```

- name: SonarCloud - Begin Analysis
  env:
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
  run: |
    dotnet sonarscanner begin \
      /k:&quot;${ vars.SONAR_PROJECT_KEY }&quot; \
      /o:&quot;${ vars.SONAR_ORGANIZATION }&quot; \
      /d:sonar.token=&quot;${SONAR_TOKEN}&quot; \
      /d:sonar.host.url=&quot;https://sonarcloud.io&quot;;

- name: Restore
  run: dotnet restore

- name: Build
  run: dotnet build --no-restore --configuration Release

- name: Test
  run: |
    dotnet test \
      --no-build \
      --configuration Release \
      --collect:&quot;XPlat Code Coverage&quot; \
      --logger &quot;trx&quot;;

- name: SonarCloud - End Analysis
  env:
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
  run: |
    dotnet sonarscanner end \
      /d:sonar.token=&quot;${SONAR_TOKEN}&quot;;

```

Configuration

Import/bind the GitHub repository correctly in SonarCloud. Use the actual organization/project keys. Store the Sonar token as a GitHub secret.

Quality Gate

Analysis and Quality Gate are distinct. The gate applies configured criteria such as zero new bugs or a minimum New Code Coverage.

Lesson 10 — Artifacts & Build Once, Deploy Many

Artifact

An artifact is persisted build output consumed later by jobs or deployment workflows.

Upload

```

- name: Publish application
  run: dotnet publish --configuration Release --output ./publish

- name: Upload application artifact
  uses: actions/upload-artifact@v4
  with:
    name: employee-api
    path: ./publish

```

Download

```

- name: Download application artifact
  uses: actions/download-artifact@v5
  with:
    name: employee-api
    path: ./publish

```

Build Once, Deploy Many

```

Build
↓
Artifact A
↓
Dev → QA → Production

```

Do not rebuild independently for each environment.

Artifact vs NuGet

A pipeline artifact is a CI/CD handoff. A NuGet package is normally a reusable .NET library distributed through a package feed.

Lesson 11 — Complete Enterprise CI

Responsibility

CI answers: Is the source buildable, testable, analyzed, and packaged into a deployable artifact?

Flow

```
Checkout
↓
.NET setup
↓
Restore
↓
Sonar Begin
↓
Build
↓
Unit Tests
↓
Coverage
↓
Publish Test Results
↓
Publish Coverage
↓
Sonar End / Quality Gate
↓
dotnet publish
↓
Upload Artifact
```

PR vs main CI

PR validation checks proposed changes before merge. Main CI validates the exact merged commit that becomes deployable. Re-running tests, coverage, and Sonar in main CI is reasonable.

Principles

Fail fast for correctness checks. Allow reporting to run after failures when useful. Produce an immutable artifact. Keep commit → CI run → artifact → deployment traceability.

Lesson 12 — CD Fundamentals

Responsibility

CD consumes the validated artifact and promotes it through environments. It should not independently rebuild.

Architecture

```
successful CI
↓
CD workflow
↓
Download exact artifact
↓
Development
↓
QA
↓
Production
```

GitHub Environments

Environments can provide environment-specific variables/secrets, required reviewers, deployment protection, branch restrictions, and deployment history.

needs vs environment

```
needs: deploy-dev
  → dependency/order

environment: Production
  → environment configuration/protection
```

Approval

Production can require reviewers; the deployment job waits for environment approval.

Lesson 13 — Azure App Service Deployment

Authentication

The modern pattern is GitHub Actions → OIDC → Microsoft Entra ID → Azure RBAC → App Service.

Azure Login

```
permissions:
  id-token: write

- name: Azure Login
  uses: azure/login@v2
  with:
    client-id: ${ vars.AZURE_CLIENT_ID }
    tenant-id: ${ vars.AZURE_TENANT_ID }
    subscription-id: ${ vars.AZURE_SUBSCRIPTION_ID }
```

Deployment

```
- name: Deploy to Azure Web App
  uses: azure/webapps-deploy@v3
  with:
    app-name: ${ vars.APP_NAME }
    package: ./publish
```

Permission distinction

```
contents: read    → Git repository contents
actions: read     → GitHub Actions resources/artifacts
packages: write   → GitHub Packages
id-token: write   → OIDC token
```

Least privilege

For pure artifact-based CD, avoid checkout when source files are not needed.

Lesson 14 — Complete CD Workflow

Purpose

Before introducing reuse, explicitly show the complete Dev → QA → Production deployment logic.

Reference CD

```
name: Employee API - CD

on:
  workflow_run:
    workflows:
      - "Employee API - CI"
    types:
      - completed

jobs:
  deploy-dev:
    if: ${ github.event.workflow_run.conclusion == &#x27;success&#x27; }
    runs-on: ubuntu-latest
    environment: Development
    permissions:
      actions: read
      id-token: write
```

```

steps:
- name: Download application artifact
  env:
    GH_TOKEN: ${ github.token }
    RUN_ID: ${ github.event.workflow_run.id }
  run: |
    mkdir -p ./publish
    gh run download "${RUN_ID}" \
      --repo "${ github.repository }" \
      --name employee-api \
      --dir ./publish
- name: Azure Login
  uses: azure/login@v2
  with:
    client-id: ${ vars.AZURE_CLIENT_ID }
    tenant-id: ${ vars.AZURE_TENANT_ID }
    subscription-id: ${ vars.AZURE_SUBSCRIPTION_ID }
- name: Deploy to Azure Web App
  uses: azure/webapps-deploy@v3
  with:
    app-name: ${ vars.APP_NAME }
    package: ./publish

deploy-qa:
  needs: deploy-dev
  runs-on: ubuntu-latest
  environment: QA
  permissions:
    actions: read
    id-token: write
  steps:
    # Same artifact download, Azure login, and deployment logic

deploy-prod:
  needs: deploy-qa
  runs-on: ubuntu-latest
  environment: Production
  permissions:
    actions: read
    id-token: write
  steps:
    # Same artifact download, Azure login, and deployment logic

```

GH_TOKEN

GH_TOKEN is an environment variable recognized by the GitHub CLI. It is populated from github.token so gh run download can authenticate.

Exact artifact

github.event.workflow_run.id identifies the CI run that triggered CD, allowing CD to target the artifact from that run.

Lesson 15 — Reusable Workflows & Templates

Why reuse?

The unrefactored CD workflow duplicates Download → Azure Login → Deploy three times. Reusable workflows centralize implementation and accept inputs.

Reusable workflow

```

name: Reusable Azure Deployment

on:
  workflow_call:
    inputs:
      environment-name:
        required: true
        type: string
      app-name:
        required: true
        type: string

```

Caller

```

jobs:
  deploy-dev:
    uses: ../github/workflows/deploy.yml

```

```

with:
  environment-name: Development
  app-name: EmployeeApi-Dev

deploy-qa:
  needs: deploy-dev
  uses: ../github/workflows/deploy.yml
  with:
    environment-name: QA
    app-name: EmployeeApi-QA

```

Important distinction

A reusable workflow is called at the job level, not inserted as a normal step.

Configuration separation

Keep deployment logic in the reusable workflow; keep environment-specific values and protection rules in GitHub Environments.

Composite action

Composite actions package reusable steps. Reusable workflows are better for reusable CI/CD processes involving jobs, permissions, and environments.

Lesson 16 — Matrix Strategy & Parallelism

Purpose

Matrix runs the same job against multiple configurations such as .NET versions or operating systems.

Example

```

strategy:
  matrix:
    dotnet-version:
      - &#x27;8.0.x&#x27;
      - &#x27;9.0.x&#x27;
      - &#x27;10.0.x&#x27;

steps:
- uses: actions/setup-dotnet@v4
  with:
    dotnet-version: ${{ matrix.dotnet-version }}

```

Multiple dimensions

```

strategy:
  matrix:
    os:
      - ubuntu-latest
      - windows-latest
    dotnet-version:
      - &#x27;8.0.x&#x27;
      - &#x27;9.0.x&#x27;

```

This creates $2 \times 2 = 4$ combinations.

Matrix vs needs

matrix multiplies configurations; needs defines dependencies/order.

Controls

fail-fast can cancel other matrix executions after failure. max-parallel limits simultaneous matrix jobs.

Don't use Matrix for sequential environments

Dev → QA → Production should use needs, not a matrix that encourages parallel deployment.

Good use

Use matrix when multiple versions/platforms are actual compatibility requirements.

Lesson 17 — NuGet Caching & Packages

NuGet caching

Caching avoids repeatedly downloading dependencies. setup-dotnet supports NuGet caching directly.

GitHub cache

```
- name: Setup .NET
  uses: actions/setup-dotnet@v4
  with:
    dotnet-version: &#x27;8.0.x&#x27;
    cache: true

- name: Restore
  run: dotnet restore
```

Cache vs feed

A cache is a performance optimization. A package feed stores and distributes versioned packages.

Azure Artifacts

Azure DevOps provides Azure Artifacts feeds, including NuGet. Use dotnet pack to create a .nupkg and push it to an internal feed.

Azure DevOps example

```
- task: DotNetCoreCLI@2
  displayName: &#x27;Create NuGet Package&#x27;
  inputs:
    command: &#x27;pack&#x27;
    packagesToPack: &#x27;*/Employee.Common.csproj&#x27;
    configuration: &#x27;Release&#x27;
    outputDir: &#x27;$(Build.ArtifactStagingDirectory)/nuget&#x27;

- task: DotNetCoreCLI@2
  displayName: &#x27;Publish NuGet Package&#x27;
  inputs:
    command: &#x27;push&#x27;
    packagesToPush: &#x27;$(Build.ArtifactStagingDirectory)/nuget/*.nupkg&#x27;
    nuGetFeedType: &#x27;internal&#x27;
    publishVstsFeed: &#x27;MyProject/MyFeed&#x27;
```

GitHub Packages

GitHub Packages can host NuGet packages associated with GitHub.

GitHub example

```
name: Publish NuGet Package

on:
  push:
    branches:
      - master

jobs:
  publish-package:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      packages: write
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with:
          dotnet-version: &#x27;8.0.x&#x27;
          cache: true
      - run: dotnet restore
      - run: dotnet build --no-restore --configuration Release
      - run: dotnet test --no-build --configuration Release
      - name: Create NuGet package
        run: |
```

```
dotnet pack src/Employee.Common/Employee.Common.csproj \
  --no-build --configuration Release --output ./packages
- name: Publish NuGet package
  run: |
    dotnet nuget push "./packages/*.nupkg" \
      --source "https://nuget.pkg.github.com/${{ github.repository_owner }}/index.json" \
      --api-key "${{ secrets.GITHUB_TOKEN }}" \
      --skip-duplicate
```

Permissions

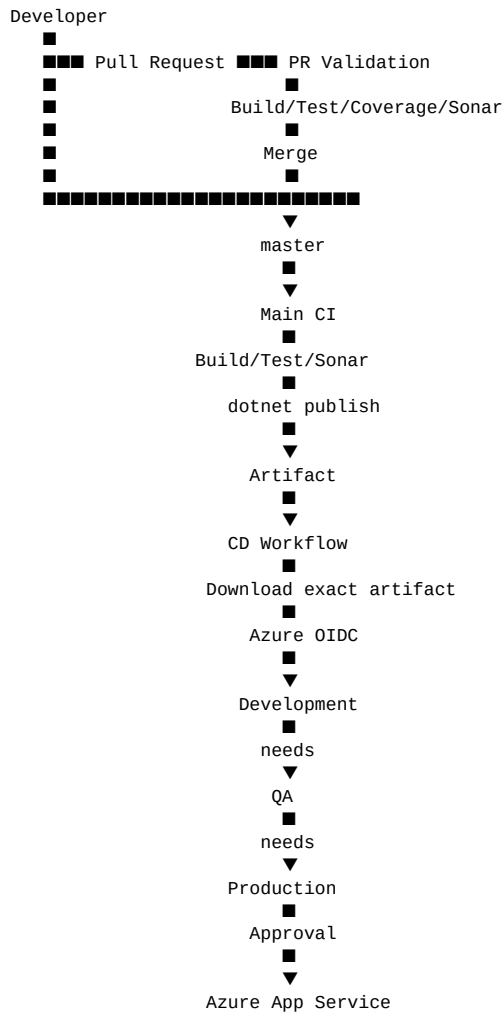
```
contents: read
packages: write
```

contents: read is repository access. packages: write allows publishing to GitHub Packages. id-token: write is for OIDC/Azure.

Application vs library

EmployeeManagementAPI is normally deployed with dotnet publish. Reusable libraries such as Employee.Common can be packed and published to a NuGet feed.

Consolidated Enterprise Architecture



Core rule: CI creates the deployable artifact; CD consumes and promotes that artifact. Do not rebuild independently for each environment.

GitHub / SonarCloud / Azure Checklist

Area	Configuration	Purpose
GitHub	Workflow permissions	Limit GITHUB_TOKEN capabilities
GitHub Actions	SONAR_TOKEN secret	Authenticate SonarCloud
GitHub Actions	SONAR_ORGANIZATION / SONAR_PROJECT_KEY	Identify Sonar project
SonarCloud	GitHub repository binding	Correct repository/PR association
GitHub Environments	Development / QA / Production	Configuration and protection
GitHub Environments	APP_NAME	Environment-specific App Service
Azure / Entra ID	Federated credential	Trust GitHub OIDC
Azure	RBAC role assignment	Permit deployment
GitHub	Production reviewers	Approval before production
GitHub Packages	packages: write	Publish NuGet package

Developer-Focused Interview Q&A;

Q: What is GitHub Actions?

A: A GitHub-native automation platform for CI, CD, testing, packaging, security checks, and repository automation.

Q: What is a workflow?

A: A YAML-defined automation pipeline triggered by GitHub events.

Q: What is a runner?

A: The machine or execution environment that runs a job.

Q: uses vs run?

A: uses invokes an action; run executes shell commands.

Q: needs vs if?

A: needs controls dependency/order; if controls whether a job or step executes.

Q: Why artifacts?

A: To persist build output and transfer a specific deployable version between pipeline boundaries.

Q: Why Build Once, Deploy Many?

A: Dev, QA, and Production receive the same validated binary, improving reproducibility and traceability.

Q: Why Environments?

A: They provide environment-specific configuration and protection such as secrets, variables, approvals, and deployment history.

Q: What is OIDC?

A: A federation mechanism that lets GitHub Actions obtain short-lived cloud identity tokens.

Q: What does id-token: write mean?

A: The workflow can request an OIDC identity token.

Q: What does contents: read mean?

A: The GITHUB_TOKEN can read repository contents.

Q: What does packages: write mean?

A: The GITHUB_TOKEN can publish packages to GitHub Packages.

Q: What is workflow_run?

A: A trigger that runs a workflow after another workflow completes and exposes information about that run.

Q: Why workflow_run for CD?

A: It can ensure CD starts after CI completion and lets CD identify the CI run whose artifact should be deployed.

Q: What is a reusable workflow?

A: A workflow exposed with workflow_call so another workflow can call it with inputs/secrets.

Q: Reusable workflow vs composite action?

A: Reusable workflows reuse jobs and orchestration; composite actions package reusable steps.

Q: What is Matrix?

A: A strategy for running a job against multiple configurations such as OS or .NET versions.

Q: Should Dev/QA/Prod be a matrix?

A: Usually no when promotion must be sequential; use needs.

Q: NuGet cache vs feed?

A: Cache speeds restore; a feed stores and distributes versioned packages.

Q: What is dotnet pack?

A: It creates a NuGet package, normally a .nupkg, for a reusable .NET library.

Q: Azure Artifacts vs GitHub Packages?

A: Both host packages such as NuGet; Azure Artifacts belongs to Azure DevOps and GitHub Packages belongs to GitHub.

Q: Why publish test results in main CI if PR validation does it?

A: PR validation checks proposed changes; main CI validates the merged commit that becomes the deployable artifact.

Q: Why avoid long-lived Azure credentials?

A: OIDC reduces credential exposure and rotation burden by using short-lived identity tokens.

Cheat Sheet

Concept	Remember
Workflow	Pipeline definition
on	When workflow starts
Job	Logical execution unit
runs-on	Runner image
steps	Sequential operations in a job
uses	Invoke action
run	Execute shell command
needs	Dependency/order
if	Conditional execution
env	Environment variables
vars	Non-sensitive configuration
secrets	Sensitive configuration
environment	Deployment configuration/protection
artifact	Persisted build output
workflow_call	Reusable workflow entry point
matrix	Multiple configurations
id-token: write	OIDC token permission
contents: read	Repository-content permission
actions: read	Read GitHub Actions resources/artifacts where required
packages: write	Publish GitHub Packages
dotnet pack	Create NuGet package
dotnet publish	Create deployable application output
cache: true	Enable NuGet caching in setup-dotnet